Divide and conquer

CS3230 · Lecture 3

Demo course material, written for OpenPapr

Recap

- Recurrences describe recursive running times
- Today: designing recursive algorithms

Divide, conquer, combine

- Divide the problem into smaller parts
- Conquer each part recursively
- Combine the answers

Example: merge sort

- Split the array in half
- Sort each half recursively
- Merge the two sorted halves

Merging

- Walk both halves with two pointers
- Take the smaller head each time
- Theta(n) for n elements in total

Correctness

- Induction on the array length
- Base case: one element is sorted
- Merging two sorted halves gives a sorted array

Recursion tree

- Each level does Theta(n) work in total
- Sizes n, n/2, n/4, ..., 1
- log2 n + 1 levels

Merge sort as a recurrence

- $T(n) = 2T(n/2) + \Theta(n)$
- Solves to $T(n) = \Theta(n \log n)$
- Matches the recursion tree

Example: binary search

- T(n) = T(n/2) + Theta(1)
- T(n) = Theta(log n)

Example: Karatsuba

- Multiply two n-digit numbers with 3 half-size products, not 4
- T(n) = 3T(n/2) + Theta(n)
- Theta(n^1.585) instead of Theta(n^2)

When is it worth it?

- The parts must be independent
- Combining must be cheap
- Overlapping parts: try dynamic programming

Summary

- Divide, conquer, combine
- Write the recurrence, then solve it
- Next: the master theorem